# Лекція 3
# РЕГУЛЯРНІ ВИРАЗИ

Синтаксис, r-рядки, модуль r

# Регулярні вирази

**Регулярний вираз** — це спеціальний рядок, який описує **шаблон** для пошуку в тексті.

Використовуються при:

✓ Роботі з текстами (очищення, форматування).

✓ Перевірці введених даних (email, телефони).

✓ Веб-скрапінгу та парсингу даних.

✓ Аналізі логів та системних звітів.

✓ Фільтрації контенту та безпеці.

✓ Автоматизації обробки коду.

# Шаблони та метасимволи

Використання регулярних виразів при роботі з текстом полягає у створенні шаблонів, які виявляють (match) або змінюють частини рядків.

**Шаблон регулярного виразу** – це рядок, який містить звичайні та мета- символи.

**Метасимволи** – це спеціальні символи, які мають особливе значення в регулярних виразах і використовуються для створення гнучких шаблонів пошуку.

Якщо звичайні символи (a, b, 1, @) просто шукаються у тексті, метасимволи керують процесом пошуку та визначають складні правила.

# regex101.com

**Для роботи з регулярними виразами існує багато засобів, одним із них може бути сайт regex101.com .**

SCAN ME

# Метасимволи, що відповідають багатьом символам

| Метасимвол | Опис | Приклад |
|---|---|---|
| **.** | Будь-який символ, крім нового рядка (\n) | **c.t** відповідатиме, наприклад, таким фразам **cat**, **c*t**, **cUt**, **c3t** |
| **\d** | Відповідає будь-якій цифрі від 0 до 9 | **\d\d** буде знаходити всі двоцифрові фрагменти. Наприклад, **65**, **02** |
| **\D** | Заперечення \d, відповідає будь-якому символу, що не є цифрою | **\d\D** шукатиме фрагмент, де підряд ідуть цифра та не цифра. Наприклад, **4a**, **5_** |
| **\w** | Будь-який буквено-цифровий символ (**a-z**, **A-Z**, **0-9**, _) | **\wox** може відповідати за **fox**, **_ox**, **4ox** |
| **\W** | Заперечення до \w | **\d\W** відповідатиме, наприклад, таким фрагментам: **7%**, **2!**, **4˚** |
| **\s** | Відповідає символу пропуску (включно з табуляцією), але не відповідає за перехід на новий рядок | **1\s6** може відповідати таким фрагментам тексту: **1 6** або<br>**1 6** |
| **\S** | Заперечення до \s | **1\S6** відповідатиме, наприклад, 116, 1a6 |

# Метасимволи, що відповідають локації

| Метасимвол | Опис | Приклад |
|---|---|---|
| **^** | Позначає початок рядка | **^Hello** знайде **Hello**, якщо це слово розміщено на початку рядка |
| **$** | Позначає кінець рядка | **world$** знайде **world**, якщо це слово стоїть у кінці рядка |
| **\b** | Відповідає за межі слова, тобто початок або кінець (\B – заперечення) | У реченні **Find a cat in this category** регулярний вираз cat відповідатиме за **cat** та **cat**egory, а **\bcat\b** знайде тільки слово **cat** |

Який регулярний вираз знайде всі слова, що закінчуються на "ing"?

Який регулярний вираз знайде всі слова, що закінчуються на "ing"?

**\w+ing\b**

# Метасимволи, що відповідають багатьом символам

| Метасимвол | Опис | Приклад |
|---|---|---|
| * | Нуль і більше повторень шаблону, що передує символу * | **tan*** відповідатиме, наприклад, таким фразам: **ta**lk, **tan, tan**k, **tan**dem, **tan**go, ka**tan**a, s**ta**ck |
| + | Одне або більше повторень шаблону | Якщо у попередньому прикладі замінити у шаблоні * на +, то отримаємо такий результат: **tan+** відповідатиме фразам: talk, **tan, tan**k, **tan**dem, **tan**go, ka**tan**a, stack |
| ? | 0 або 1 повторення шаблону | **студенти?** Відповідатиме і за **студент**, і за **студенти**, а **colou?r** за **color** та **colour** |
| {N} | N повторень шаблону, де N – натуральне число | **a\d{2}** може відповідати за **a54**, **a06**, тощо |
| {N,M} | Від N до M повторень шаблону, де N, M – натуральні числа | **a\d{2,3}** відповідатиме, наприклад, таким фрагментам: **a12, a763** |

# «Жадібні» символи

Симовли +, *, ? Називають ще «жадібними» (greedy), бо вони завжди намагаються захопити рядок якомога більшої довжини:

Рядок: abababababacc

Шаблон: **a\w*a**

Результат: **abababababa**cc

Для того, щоб шаблон охоплював якомога  меншу кількість символів, доцільно після «жадібного» символа додати ?.

Шаблон: **a\w*?a**

Результат: **aba**b**aba**b**aba**cc

# «Жадібні» символи

Зауважимо, що *,?, + сприймаються компілятором як метасимволи, а якщо потрібно знайти котрийсь із них конкретно у рядку, то доцільно скористатись ескейп-послідовностями:

Рядок: 12+2=14

Шаблон1: **\d+**

Результат: 12**+**2**=**14

Шаблон2: **\d\+**

Результат: 1**2+2=14**

Шаблон2: **\d+\+**

Результат: ?????

# «Жадібні» символи

Зауважимо, що *,?, + сприймаються компілятором як метасимволи, а якщо потрібно знайти котрийсь із них конкретно у рядку, то доцільно скористатись ескейп-послідовностями:

Рядок: 12+2=14

Шаблон1: **\d+**

Результат: 12**+**2**=**14

Шаблон2: **\d\+**

Результат: 1**2+2=14**

Шаблон2: **\d+\+**

Результат: 12+2=14

Як записати регулярний вираз для пошуку номерів телефону у форматі 050-123-45-67?

Як записати регулярний вираз для пошуку номерів телефону у форматі 050-123-45-67?

\d{3}-\d{3}-\d{2}-\d{2}

# Задання множини символів

- Щоб задати множину допустимих символів у певному місці шаблону потрібно скористатись квадратними дужками [ ].
- У квадратних дужках можна перерахувати символи, що можуть стояти на цій позиції, або задати певний діапазон (наприклад, C-Q, a-z, 3-8).
- Інколи кажуть, що [ ] задають класи символів, але на практиці це не має нічого спільного із класами ООП.

- Для того, щоб визначити заперечення наявності певних символів на вказаній позиції використовують синтаксис [^].

# Задання множини символів

Рядок: ema, cat, cбt, cutlery, e e

Шаблон1: c[aeiouб,]t

Результат: ema, cat, cбt, cutlery, e a

Шаблон2: e[^aeiou]a

Результат: ema, cat, cбt, cutlery, e a

# Вибір з декількох варіантів

Часто потрібно задавати шаблони, які передбачають, що на певній позиції може знаходитись декілька варіантів «підшаблонів». Щоб це реалізувати можна скористатись оператором |.

Наприклад, шаблон **__|\d\d** відповідає двом підкресленням або двом цифрам (**__**, **24**, **18** тощо).

А шаблон **\bM(r|rs|iss|s)\.?** знайде такі фрагменти тексту:

**Mr. Smith and Mrs. Johnson had a meeting with Miss Taylor and Ms. Doe.**

Слід зауважити, що при зчитуванні шаблону Python перевіряє варіанти зліва на право.

# Особливості задання шаблонів у Python

У Python шаблони регулярних виразів – це звичайні рядки. Як ми вже бачили, деякі із метасимволів регулярних виразів є тотожними до синтаксису escape-послідовностей (наприклад, \b – обрамлення слова у регулярних виразах і \b – аналог backspace у escape-послідовностях).

Тому, щоб розрізняти ці випадки для регулярних виразів бекслеш потрібно дублювати (\\b, \\w, \\s )

Щоб уникнути казусів та зайвого дублювання часто використовують «сирі» (raw) рядки (r-strings).

r-рядки позначають літерою r перед лапками для оголошення рядка.

'this is a string'
r'this is a raw string'

# Модуль re. Знаходження всіх входжень шаблону у тексті

Модуль **re** – це стандартна бібліотека Python для роботи з **регулярними виразами (RegEx)**.

Він дозволяє **шукати, замінювати, розбивати текст** за певними шаблонами.

Найпростіший варіант застосування – знаходження всіх частин тексту, які відповідають заданому шаблону. Для цього існує функція:

**re.findall(pat, s),**

де pat – шаблон,  s – текст, до якого застосовується шаблон.

У результаті, функція повертає список підрядків, що відповідають шаблону.

```
re
├── Pattern (скомпільований шаблон)
│   ├── match()
│   ├── search()
│   ├── findall()
│   ├── finditer()
│   ├── sub()
│   ├── split()
│   ├── fullmatch()
│   ├── subn()
│
├── Match (об'єкт результату пошуку)
│   ├── group()
│   ├── start()
│   ├── end()
│   ├── span()
│   ├── groups()
│   ├── groupdict()
│
├── Iterator (ітератор для `finditer()`)
│   ├── __iter__()
│   ├── __next__()
│
└── error (виняток `re.error`)
```

# Модуль re. Знаходження всіх входжень шаблону у тексті

Нехай задано файл, у якому записані деякі роки трьома або чотирма цифрами. Потрібно знайти ці роки, а також номери рядків, де вони записані.

Для вирішення цієї задачі скористаємось шаблоном

**pat = r'\d{3,4}'**

А повернення списку років реалізуємо скориставшись **re.findall()**.

Розв'язання (regex1.py)

# Компіляція регулярних виразів

Якщо деякий шаблон потрібно використовувати дуже багато разів у програмі, то для пришвидшення виконання цей шаблон доцільно прокомпілювати. Для цього модуль re надає функцію

**re.compile(pat, flags=0),**

де *pat* – шаблон, а *flags* – опціональний параметр, який є спеціальним модифікатором, що застосовується до шаблону.

У результаті виконання функції *re.compile()* повертається відкомпільований шаблон (як об’єкт класу re.Pattern), який використовується для обробки тексту.

# Компіляція регулярних виразів

Розглянемо детальніше деякі можливі значення параметра flags.

| Значення flags | Опис |
|---|---|
| re.IGNORECASE (re.I) | Ігнорує регістр (не розрізняє a і A) |
| re.MULTILINE (re.M) | Дозволяє ^ і $ працювати для кожного рядка окремо |
| re.DOTALL (re.S) | Символ . охоплює також \n |
| re.VERBOSE (re.X) | Дозволяє додавати пробіли і коментарі (починаються із символа #) для покращення читабельності |

# Компіляція регулярних виразів

```python
import re

pattern = re.compile(r'\d+')  # Шукаємо всі натуральні числа
text = "У мене 3 яблука і 12 груш."

matches = pattern.findall(text)
print(matches)  # ['3', '12']

```

Компіляція без прапорців

```python
import re

pattern = re.compile(r'python', re.IGNORECASE)
text = "I love Python and PYTHON is great!"

matches = pattern.findall(text)
print(matches)  # ['Python', 'PYTHON']


```

Використання re.IGNORECASE

# Компіляція регулярних виразів

На практиці часто постають задачі, коли потрібно використати одразу декілька прапорців для компіляції деякого шаблону. Тоді можна скористатись оператором | для їх об’єднання.

Розглянемо використання цього оператора на приклад задачі для знаходження всіх заголовків деякого тексту.

File Edit View Navigate Code Refactor Run Tools VCS Window Help pr_list - metodychka.py

pr_list metodychka.py

metodychka.py

```python
text = """# Заголовок 1
Текст текст текст.

# Заголовок 2
Ще один текст.

# Останній заголовок
Фінальний абзац."""

# Код для компіляції без додавання коментарів
# pattern = re.compile(r"^\#\s*(.+)", re.MULTILINE)

# Компілюємо регулярний вираз з декількома прапорцями
pattern = re.compile(r"""
    ^\#\s*        # Початок рядка, далі символ '#' (заголовок)
    (.+)          # Будь-який текст після '#'
""", re.MULTILINE | re.VERBOSE)

# Знаходимо всі заголовки
matches = pattern.findall(text)

# Виводимо результат
print(matches)
```

Run: metodychka

```
['Заголовок 1', 'Заголовок 2', 'Останній заголовок']

Process finished with exit code 0
```

Version Control Run Python Packages TODO Python Console Problems Terminal Services

Indexing completed in 9 min, 8 sec. Shared indexes were applied to 29% of files (1,998 of 6,772). (7 minutes ago)

1:1 CRLF UTF-8 4 spaces Python 3.11

# Функції модуля re та методи класу regex для пошуку підрядків

Більшість функцій модуля re можна використовувати також і як методи екземплярів класу re.Pattern, але для цього, очевидно, потрібно скомпілювати шаблон.

Розглянемо детальніше деякі з цих функцій, вважаючи, що *rgx* – це вже відкомпільований шаблон, а *pat* – звичайний шаблон у вигляді рядка.

# Функції модуля re та методи класу regex для пошуку підрядків

| Функція (метод) | Опис |
|---|---|
| re.search(pat, string, flags = 0)<br>або<br>rgx.search(string) | Знаходить перший підрядок, що відповідає шаблону, у рядку string. Повертає об’єкт класу re.Match або None, якщо нічого не знайдено. |
| re.mathch(pat, string, flags = 0)<br>або<br>rgx.smatch(string) | Перевіряє, чи відповідає початок рядка string шаблону. Повертає об’єкт класу re.Match або None, якщо не відповідає. |
| re.findall(pat, string, flags = 0)<br>або<br>rgx.findall(string) | Повертає список усіх підрядків рядка string, що відповідають заданому шаблону. |
| re. finditer(pat, string, flags = 0)<br>або<br>rgx.finditer(string) | Повертає ітератор, який пробігає послідовність усіх підрядків рядка string, що відповідають шаблону. Кожен елемент отриманої ітерованої послідовності є екземпляром класу re.Match. |

# Клас re.Match

Клас re.Match дозволяє визначати характер підрядка, який відповідав шаблону. Як ми вже бачили, об'єкти цього класу створюються у результаті виконання функцій (методів) search, match, finditer.

Розглянемо методи класу re.Match вважаючи, що mt – це деякий екземпляр цього класу.

| Метод | Опис |
|---|---|
| mt.group() | Повертає підрядок, що відповідає шаблону |
| mt.start() | Повертає початкову позицію у рядку підрядка, що відповідає шаблону |
| mt.end() | Повертає кінцеву позицію у рядку підрядка, що відповідає шаблону |
| mt.span() | Повертає кортеж із початкової та кінцевої позицій у рядку підрядка, що відповідає шаблону |

# Клас re.Match

Розглянемо такий приклад.

У текстовому файлі міститься текст, що включає інформацію про роки, коли відбулися певні події.

Побудувати впорядкований список років з цього тексту разом із номерами рядків файлу, де вказано той чи інший рік.

Крім того, вказати також позицію цього року у самому рядку та у цілому файлі.

Повернення дат реалізувати з використанням функції finditer(), а аналіз характеристик провести засобами класу re.Match.

Розв’язання. (regex1_2.py)

# Клас re.Match

Розглянемо ще один приклад.

Деякий навчальний заклад у текстовому файлі збирає інформацію про студентів.

У файлі містяться дані про: ПІП студента, номер студентського квитка, дату народження, телефон.

Побудувати словник, ключами якого є номери студентських квитків, а значеннями – відповідна інформація про студентів.

Перевірити правильність інформації.

Очікуваний формат даних у файлі:

Прізвище Ім’я По-батькові АБ12345678 01.01.2005 +380 (00) 000 0000

Розв’язок. (regex2.py)

# F & Q